MUSEU PALEO · ESTUDO COMPARATIVO

O QUE O PHP FAZIA NO PROJETO

Comparação entre a versão com PHP (branch `main`) e a versão só com HTML, CSS e JavaScript (branch `versao-sem-php`).

Branch	Pasta no seu computador	Tecnologias
`main`	`landing-page/museu`	PHP + HTML + CSS + JavaScript
`versao-sem-php`	`landing-page/museu-sem-php`	só HTML + CSS + JavaScript

Visualmente as duas são **iguais**. A diferença está em **onde** as coisas acontecem e em **quanto trabalho dá manter**.

1 A IDEIA PRINCIPAL

- **PHP roda no servidor.** Antes de a página chegar ao navegador, o servidor executa o PHP, monta o HTML e só então envia o resultado pronto.
- **JavaScript roda no navegador.** A página chega como está, e o JS do visitante faz as mudanças depois.
- **HTML puro não "pensa".** É um arquivo fixo: o que está escrito é o que aparece.

```
Versão PHP:      navegador ──pede──▶ servidor (roda PHP, monta HTML) ──envia HTML pronto──▶ navegador
Versão sem PHP:  navegador ──pede──▶ servidor (só entrega o arquivo)   ──envia arquivo─────▶ navegador (JS faz o resto)
```

2 AS FUNÇÕES DO PHP NESTE PROJETO

2.1 Reaproveitar o cabeçalho e o rodapé (include)

COM PHP

O topo (logo, busca, menu) e o rodapé ficam em `includes/header.php` e `includes/footer.php`. Cada página puxa cada um com uma linha:

```
<?php require __DIR__ .
'/includes/header.php'; ?>
```

SEM PHP

O cabeçalho (cerca de 35 linhas) e o rodapé (cerca de 32 linhas) estão **copiados nas 4 páginas**.

Para trocar um item do menu, é preciso editar 4 arquivos e torcer para não esquecer nenhum.

> Foi o ganho mais visível do PHP: **um lugar só para mudar**.

2.2 Um arquivo central com todo o conteúdo (includes/data.php)

COM PHP

Horários, preços, as 12 peças do acervo, a linha do tempo, os cards e o carrossel ficam em **listas** no `data.php`. As páginas usam `foreach` para gerar o HTML:

```
<?php foreach ($ACERVO as $peca): ?>
  <figure>... <?= e($peca['titulo']) ?> ...
</figure>
<?php endforeach; ?>
```

Peça nova = **uma linha**. Ela aparece na galeria, no filtro, na contagem e na busca ao mesmo tempo.

SEM PHP

As 12 peças estão escritas uma a uma no `exposicoes.html`.

Para adicionar uma peça, é preciso copiar um bloco de HTML inteiro e, para ela aparecer na busca, adicionar também no `dados.js`.

A mesma informação existe em dois lugares, e é fácil um ficar diferente do outro.

2.3 "Aberto agora / Fechado" e o dia de hoje

COM PHP

A função `status_agora()` olha o relógio **do servidor**, no fuso de Brasília, e já escreve "Aberto agora · até 18:00" no HTML. O dia de hoje vem com o selo **HOJE**.

SEM PHP

A página chega com "Verificando horário…", e o JavaScript calcula com o relógio **do visitante**, usando o fuso `America/Sao_Paulo`.

Depende de o relógio do visitante estar certo, e por uma fração de segundo aparece o texto provisório.

2.4 O buscador

COM PHP

O `search.php` procura no conteúdo do `data.php` e devolve os resultados.

Funciona até sem JavaScript, e o índice é **gerado automaticamente**: mudou um preço, a busca já sabe.

SEM PHP

O índice inteiro (47 itens) fica no `dados.js` e é baixado por **todo visitante**. A procura roda no navegador.

Para um site pequeno, funciona muito bem. Para milhares de páginas, ficaria pesado. E o índice **não se atualiza sozinho**.

2.5 Segurança ao mostrar textos (htmlspecialchars)

COM PHP

A função `e()` passa todo texto por `htmlspecialchars()`. Uma busca como `<script>` aparece como texto e não é executada.

SEM PHP

O JavaScript faz o mesmo com a função `escapar()`. A proteção continua, mas depende de lembrar de usá-la em todo lugar que mostra texto digitado pelo usuário.

2.6 O botão "Atualizar GitHub" e o Painel

COM PHP

O `sync.php` roda comandos `git` no seu computador (pull, commit, push), e o `painel.php` mostra os arquivos alterados. O botão só aparece em `localhost`.

SEM PHP

Isso não existe. Por segurança, o navegador **não pode** rodar programas no seu computador. Nessa branch, use o `sincronizar.bat` ou os comandos do git.

> Esse é o exemplo mais claro de algo que **só** um servidor consegue fazer.

2.7 Coisas menores

- **Ano do rodapé:** o PHP escrevia `date('Y')`. Sem PHP, um JS atualiza o ano.
- **Contagem dos filtros** ("Dinossauros 5"): o PHP contava as peças. Sem PHP, os números estão escritos à mão.
- **Links:** `index.php` virou `index.html`, e `search.php` virou `busca.html`.

3 COMPARAÇÃO EM NÚMEROS

	Com PHP (main)	Sem PHP (versao-sem-php)
Arquivos de página	4 páginas `.php` + 3 includes	4 páginas `.html` + `dados.js`
Linhas de código das páginas	~770 (páginas + includes, sem o painel)	~1.030 de HTML + 382 do `dados.js`
Onde fica o menu	1 arquivo (`header.php`)	copiado em 4 arquivos
Adicionar uma peça ao acervo	1 linha no `data.php`	1 bloco no HTML + 1 item no `dados.js`
Precisa de servidor especial	sim (PHP instalado)	não (abre com dois cliques)
Hospedagem grátis (GitHub Pages, Netlify)	**não funciona** no GitHub Pages	**funciona**
Botão/painel do GitHub	sim	não é possível
"Aberto agora"	calculado no servidor	calculado no navegador

4 QUANDO VALE A PENA USAR PHP

Use PHP (ou outra linguagem de servidor) quando o site precisa de algo que **o navegador não pode ou não deve fazer**:

1. **Guardar dados enviados pelo visitante:** formulário de contato, cadastro, comentários, agendamento de visita, venda de ingressos.
2. **Login e áreas restritas:** senhas e permissões precisam ser verificadas no servidor, onde o visitante não consegue mexer.

3. **Conteúdo que muda sozinho ou vem de um banco de dados:** catálogo com centenas de peças, notícias, estoque, eventos cadastrados por um painel.
4. **Esconder informações:** chaves de API, senhas de banco e regras de negócio. Tudo que vai para o navegador pode ser visto por qualquer um.
5. **Muitas páginas com a mesma estrutura:** reaproveitar cabeçalho, rodapé e modelos evita copiar e colar.
6. **Ações no servidor ou no computador:** rodar comandos, gerar PDFs, enviar e-mails, como o painel do GitHub deste projeto.

5 QUANDO NÃO É PRECISO USAR PHP

1. **Sites pequenos e informativos**, com poucas páginas que quase não mudam: portfólio, landing page, cardápio, página de evento. Este museu, sem o painel do GitHub, é um exemplo.
2. **Quando você quer hospedar de graça** e sem configurar servidor: GitHub Pages, Netlify e Vercel servem HTML/CSS/JS direto.
3. **Interações que acontecem só na tela:** carrossel, filtros da galeria, lightbox, animações, menu. Tudo isso já era JavaScript, mesmo na versão PHP.
4. **Protótipos e estudos**, quando o objetivo é aprender visual e interação.

> Um caminho comum: começar estático (HTML/CSS/JS) e só adicionar PHP quando surgir uma necessidade real, como um formulário que precisa salvar dados.

6 · RESUMO

- O PHP deste projeto serviu principalmente para **organizar**: um arquivo de dados, cabeçalho e rodapé reaproveitados, e o buscador gerado automaticamente.
- Ele também fez duas coisas que o HTML sozinho **não consegue**: calcular o horário no servidor e rodar o git pelo botão do site.
- A versão sem PHP prova que, para um site deste tamanho, **dá para viver sem PHP**, com a vantagem de abrir com dois cliques e hospedar de graça.
- O custo é **repetição**: a mesma informação em mais de um lugar e mais cuidado na hora de editar.